50

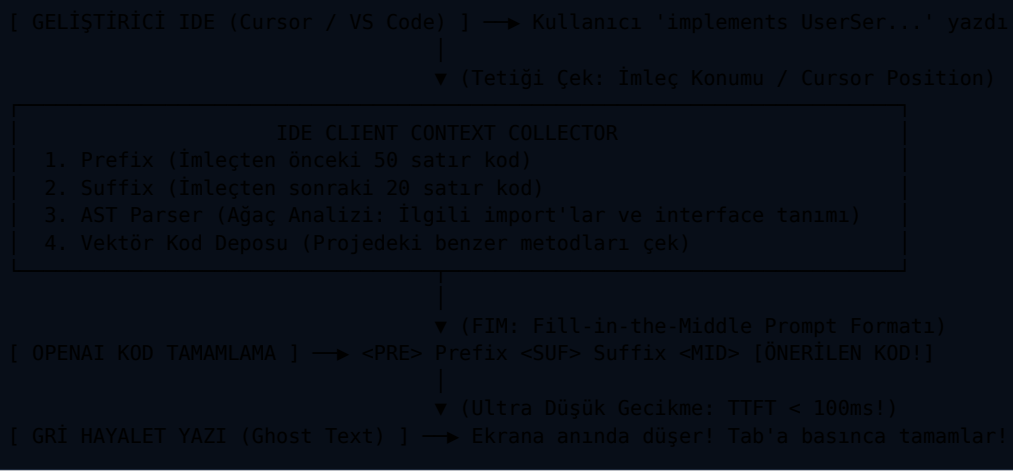
BÖLÜM: ÜRETIM DÜZEYINDE AI SISTEM TASARIMI (MAANG VAKA ÇALIŞMALARI) | MODULE: PRODUCTION AI SYSTEM DESIGN: FLAGSHIP MAANG CASE STUDIES

Vaka 2: Kod Asistanı Mimarisi (GitHub Copilot Klonu: AST, Vector & Streaming)

Case 2: Code Assistant Architecture: AST Parsing, Vector Code Indexing & Fill-In-The-Middle

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Akıllı bir çırak marangozun yanınızda durduğunu düşünün. Siz tahtayı ölçüp tam 'Bana çiv...' dediğiniz anda, çırak elinize çekici ve doğru boydaki çiviyi uzatır (Siz cümleyi bitirmeden ihtiyacınızı anlar!). GitHub Copilot tam olarak bu çıraktır: İmlecinizin nerede olduğuna bakar, yukarıda hangi sınıfları import ettiğinizi inceler (AST), projenin diğer dosyalarında nasıl kod yazdığınızı hatırlar ve siz daha klavyeye basarken ekrana açık gri renkte doğru kodu fısıldar!

🇬🇧 ENGLISH (ELI5)

Imagine a brilliant apprentice standing beside a master surgeon. Before the surgeon even finishes articulating the request, the apprentice slaps the exact scalpel size into the surgeon's palm. A real-time code assistant operates with this predictive foresight: it captures the cursor coordinates, parses the Abstract Syntax Tree (AST) of the active workspace, matches relevant library imports, and leverages Fill-in-the-Middle (FIM) prompt formatting to project ghost text within 100 milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kod Asistanı Sistem Tasarımı Gereksinimleri: 1) FIM (Fill-in-the-Middle): Standart LLM sadece sonrasını tahmin eder. Kod asistanı ise imlecin üstündeki kod (Prefix) ile altındaki kodu (Suffix) modele verir ve aradaki boşluğu (\<MID>) doldurmasını ister. 2) AST (Abstract Syntax Tree) Analizi: Düz metin aramak yerine Tree-Sitter kütüphanesiyle kod ağacı çıkarılır; çağrılan fonksiyonun interface tanımı ve import'ları bağlama eklenir. 3) Katı Gecikme SLA'sı: Geliştirici yazarken önerinin çıkması için TTFT < 150ms olmak zorundadır! Bu yüzden küçük, kod için optimize edilmiş hızlı modeller (GPT-4o-mini / StarCoder) kullanılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Code Assistant Engineering & Latency Budgets: 1) Fill-in-the-Middle (FIM): Specialized models ingest three partitioned segments: \<PRE> (pre-cursor text), \<SUF> (post-cursor text), and generate within \<MID>. This enables context-aware autocompletion inside existing method bodies. 2) Tree-Sitter AST Slicing: Rather than dumping whole files, a local daemon parses AST nodes to extract active class hierarchies, interface contracts, and imported method signatures. 3) Sub-150ms SLA: Programmers type at 5-10 characters/sec. If ghost text latency exceeds 150ms, the developer has already typed past the completion point, rendering it useless. Speculative decoding and aggressive client-side debouncing (50ms) are non-negotiable.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25: Kod Asistanı için Fill-In-The-Middle (FIM) Prompt Modeli
public record FimCodePrompt(String filePath, String prefix, String suffix) {
    public String toOpenAiFimFormat() {
        return String.format("""
            // Dosya: %s
            <PRE>
            %s
            <SUF>
            %s
            <MID>
            """, filePath, prefix, suffix);
    }
}

void main() {
    var prompt = new FimCodePrompt(
        "OrderService.java",
        "public class OrderService {
    public void cancelOrder(String id) {",
        "    }
}"
    );

    IO.println("Copilot FIM Prompt Yapısı:");
    IO.println(prompt.toOpenAiFimFormat());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Telif Hakkı ve Güvenlik Riski: Model internetten ezberlediği lisanslı (GPL) veya zararlı bir kod parçasını geliştiricinin projesine doğrudan önerebilir. Kurumsal kod asistanlarında üretilen kodun açık kaynak kütüphanelerle birebir eşleşip eşleşmediğini kontrol eden bir 'License & Secret Scanner' filtresi bulunmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Copyright & Secret Leakage: Generative code models risk spitting out memorized GPL-licensed code or accidentally echoing hardcoded AWS API tokens found in training sets. Enterprise code assistants must integrate a real-time post-generation license and secret scanner before displaying ghost text.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Fill-in-the-Middle (FIM), kodun hem öncesini hem sonrasını görerek arayı doldurur.
- AST (Tree-Sitter) analizi, sadece imlecin etrafındaki ilgili tipleri ve metotları seçer.
- Kod asistanlarında gecikme SLA'sı (TTFT \< 150ms) başarıyı belirleyen 1 numaralı faktördür.

🎯 Key Takeaways for the Interview

- Fill-in-the-Middle (FIM) generates code aware of both prefix and suffix boundaries.
- AST parsing isolates relevant imports, interfaces, and types to keep context razor-sharp.
- Sub-150ms TTFT latency is the defining engineering SLA for real-time code completion.